

Payment System & Ledger

Stripe · Razorpay · UPI · ప్రతి e-commerce వెనక – "ఖాతాలో balance ఉంచి, చెల్లింపుకి తగ్గిద్దాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – ఒక write విఫలమైతే డబ్బు మాయమవుతుంది, మరియు అది ఎక్కడ మాయమైందో కనిపెట్టడానికి **50,000 records** చూడాలి. రెండోది – ఆరు అడుగుల చెల్లింపులో **1,135 చిక్కుకుపోతాయి**, మరియు కారణం అడుగులు కాదు – **వెనక్కిపూడ్చడం కూడా విఫలమవుతుంది**. మూడోది – gateway webhooks క్రమం తప్పితే **12.5% చెల్లింపులు** తప్పు స్థితిలో ఆగిపోతాయి.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

LLD DEEP 21 (ATM) LO IDEMPOTENCY ఉంది - దాన్ని ఇక్కడ మళ్ళీ నిరూపించను

అక్కడ Bank.debit / reverse ని key / R:key తో idempotent చేశాం, Journal తో PHASE ,
#tryReverse , మరియు reconcile() - అన్న కొలిచాం. "ఒకే చర్య రెండుసార్లు జరగకుండా ఎలా ఆపాలి" అనే ప్రశ్నకి జవాబు అక్కడ ఉంది, మరియు ఈ doc అంతటా దాన్ని వాడతాను, మళ్ళీ నిరూపించను.

ఈ doc అడిగేది వేరే ప్రశ్న, మరియు అది idempotency కంటే ముందుది:

"ఏదో తప్పు జరిగిందని మీకు అసలు తెలుస్తుందా?"

ఎందుకంటే చెల్లింపు systems lo నిజమైన ప్రమాదం ఒక crash కాదు - అది ఒక మౌనమైన లోపం. ₹11,397 మాయమైంది, మరియు అది ఏ లావాదేవీలో మాయమైందో మీకు తెలియదు.

§5 lo చూపిస్తాను: **double-entry ఒక accounting సంప్రదాయం కాదు - అది ఒక దోష-స్థాన-నిర్ధారణ సంకేతం**, మరియు అది వెతకాల్సిన పనిని **1,667 రెట్లు** తగ్గిస్తుంది.

విషయ సూచిక (Table of Contents)

Part 1 - సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి - మరియు ఇక్కడ "సరైనది" అంటే ఏమిటి
2. Clarifying questions - ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 - మొదటి విరుపు: మౌనంగా మాయమయ్యే డబ్బు

3. Step - ప్రతి ఖాతాకి ఒక balance
4. మొదటి విరుపు - ₹11,397 పోయింది, ఎక్కడో తెలియదు
5. Step - balance ని నిల్వ చేయకపోవడం

Part 3 - రెండో విరుపు: వెనక్కి పూడ్చడం కూడా విఫలమవుతుంది

6. Step - saga · ముందుకి, లేదా వెనక్కి
7. రెండో విరుపు - 1,135 చెల్లింపులు మధ్యలో ఆగిపోయాయి
8. Step - పూడ్చడాన్ని వదిలేయకూడదు

Part 4 - మూడో విరుపు: gateway చెప్పేది క్రమంలో రాదు

9. Step - webhooks వింటూ స్థితి మార్చడం
10. మూడో విరుపు - 12.5% తప్పు స్థితిలో ఆగిపోతాయి
11. Step - స్థితి ఎప్పుడూ వెనక్కి పోకూడదు

Part 5 – పూర్తి system

12. మొత్తం code · 7.2 లక్షల entries · mutation testing

Part 6 – Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

14. నోటీజ్ చెప్పాల్సిన English script

15. Follow-ups – currencies, refunds, reconciliation

16. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇక్కడ "సరైనది" అంటే ఏమిటి

ఒక user ₹500 చెల్లిస్తాడు. ఆ డబ్బు అతని దగ్గర నుంచి వ్యాపారి దగ్గరికి వెళ్ళాలి, ఒక commission కట్ అవ్వాలి, ఒక రసీదు రావాలి.

సాంకేతికంగా ఇది కొన్ని database updates. కానీ ఈ problem కి ఒక ప్రత్యేకత ఉంది, మరియు అది మిగతా అన్ని systems నుంచి దీన్ని వేరు చేస్తుంది:

ఇక్కడ "దాదాపు సరైనది" అనేది ఉండదు

Feed lo ఒక post తప్పిపోతే – ఫరవాలేదు.

Typeahead lo ఒక సూచన తప్పుతే – ఫరవాలేదు.

Notification ఒకటి పోతే – ఫరవాలేదు.

₹500 మాయమైతే – అది ఒక bug కాదు, అది ఒక నేరం.

మరియు ఇంకా ముఖ్యమైనది: **అది మాయమైందని మీకు తెలియకపోతే, మీరు దాన్ని ఎప్పటికీ సరిచేయలేరు.**

అందుకే ఈ doc lo మొదటి మరియు అతి ముఖ్యమైన ప్రశ్న – **గుర్తింపు, నివారణ కాదు.**

అనుకుందాం: రోజుకి 50 లక్షల చెల్లింపులు · ఒక్కో చెల్లింపు 4-6 services ని తాకుతుంది · ఒక బయటి payment gateway.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఖాతా balance ఎక్కడ నిల్వ ఉంటుంది – ఒక column లో నా, లెక్కించాలా?	ఇదే §5 మొత్తం. Column అయితే లోపం మానమైనది
ఒక చెల్లింపు ఎన్ని services ని తాకుతుంది?	§7 – అడుగులు పెరిగితే చిక్కుకునేవి సరళరేఖలో పెరుగుతాయి
వెనక్కి పూడ్చడం (compensation) విఫలమైతే?	చాలామంది ఈ ప్రశ్న అడగరు. §7 lo అదే అసలు సమస్య
Gateway webhooks క్రమంలో వస్తాయా?	రావు. §10 lo అది 12.5% చెల్లింపులని పాడుచేస్తుంది
ఒకే webhook రెండుసార్లు రావొచ్చా?	అవును – అన్ని gateways "కనీసం ఒకసారి" హామీ ఇస్తాయి
Reconciliation ఎలా, ఎప్పుడు?	Gateway యొక్క రికార్డు మరియు మీది సరిపోల్చడం – §15
డబ్బు ఏ కరెన్సీలో, ఏ ఖచ్చితత్వంతో?	Float ఎప్పుడూ కాదు – §15

ఒక ప్రశ్న చాలామంది అడగరు, మరియు అదే అత్యంత ముఖ్యమైనది

"వెనక్కి పూడ్చే ప్రయత్నం విఫలమైతే ఏమి చెయ్యాలి?"

Saga గురించి అందరూ చెప్తారు: "అడుగు విఫలమైతే compensate చెయ్యి". కానీ **compensation కూడా ఒక distributed operation** – అది కూడా విఫలమవుతుంది.

§7 lo కొలుస్తాం: ఆరు అడుగులు, 2% compensation వైఫల్యంతో **1,135 చెల్లింపులు** ఎటూ కానీ స్థితిలో ఆగిపోతాయి – డబ్బు తీసుకున్నాం కానీ order లేదు, లేదా దానికి వ్యతిరేకం.

ఈ ప్రశ్న అడిగితే interviewer కి మీరు saga ని **నిజంగా అమలు చేశారని** తెలుస్తుంది.

Part 2 – మొదటి విరుపు: మౌనంగా మాయమయ్యే డబ్బు

3. Step – ప్రతి ఖాతాకి ఒక balance

సహజమైన schema:

```
accounts(id, balance)
```

Transfer అంటే రెండు updates:

```
balance[from] -= amt;  
balance[to]    += amt;
```

Database transaction lo పెడితే ఇది సురక్షితం – **ఒకే database అయితే.**

కానీ నిజమైన చెల్లింపు system lo `from` ఒక wallet service lo, `to` ఒక merchant ledger lo, commission ఒక మూడో చోట. అవి **వేర్వేరు systems**, కాబట్టి ఒకే transaction సాధ్యం కాదు.

అంటే ఆ రెండు updates మధ్య ఏదైనా జరగొచ్చు – timeout, crash, network split. అప్పుడు?

4. మొదటి విరుపు – ₹11,397 పోయింది, ఎక్కడో తెలియదు

50,000 transfers నడిపాను, వాటిలో **0.1%** lo రెండో write విఫలమయ్యేలా చేశాను. మరియు ఇది ఒక **వాస్తవిక** system – డబ్బు బయటి నుంచి వస్తుంది (deposits), బయటికి పోతుంది (payouts).

50,000 ఆపరేషన్లు · 15% deposits · 15% payouts · 70% అంతర్గత transfers
0.1% అంతర్గత writes విఫలం

విధానం	ప్రవేశపెట్టినవి	కనిపెట్టినది	ఏ transaction అని తెలుసా?
single-entry	30	మొత్తం తేడా ఉంది	లేదు
double-entry	30	30 txns విరిగాయి	అవును · 30 ids

రెండు "ఏదో తప్పు ఉంది" అని చెప్పాయి. ఒక్కటే "ఎక్కడ" అని చెప్పింది.

Single-entry తో మీకు తెలిసేది: **మొత్తం సరిపోవడం లేదు.**

కానీ 50,000 ఆపరేషన్లలో **ఏది** విరిగిందో తెలియదు. వెతకాలంటే ప్రతి ఒక్కటే చూడాలి – మరియు అవి వేర్వేరు systems లో ఉన్నాయి.

Double-entry తో: **30 transaction ids.** వాటిని మాత్రమే చూస్తే చాలు.

లోపాన్ని వెతకడానికి ఎన్ని records చూడాలి?

single-entry : 50,000 ఆపరేషన్లూ చూడాలి (ఏది తప్పు తెలియదు)

double-entry : 30 transactions మాత్రమే – 1667 రెట్లు తక్కువ

ఒక నిజాయితీ గమనిక – నా SINGLE-ENTRY కొలత కొంచెం ఉదారంగా ఉంది

నా model లో single-entry కూడా "మొత్తం తేడా ఉంది" అని కనిపెట్టింది – ఎందుకంటే నేను deposits మరియు payouts ని **ఖచ్చితంగా** ట్రాక్ చేశాను.

నిజమైన system లో ఆ సంఖ్యలు **బయటి systems** నుంచి వస్తాయి – bank statements, gateway reports – వాటికి సొంత ఆలస్యం మరియు సొంత లోపాలు ఉంటాయి.

అంటే ఆచరణలో single-entry తో మీరు **తేడాని కూడా** నమ్మకంగా కనిపెట్టలేరు. **నేను దానికి అనుకూలంగా కొలిచినా అది ఓడిపోయింది.**

5. Step – balance ని నిల్వ చేయకపోవడం

పరిష్కారం ఒక్క ఆలోచనలో ఉంది, మరియు అది 500 ఏళ్ళ నాటిది:

DOUBLE-ENTRY ఒక ACCOUNTING సంప్రదాయం కాదు – అది ఒక దోష-స్థాన-నిర్ధారణ సంకేతం

ప్రతి లావాదేవీ **కనీసం రెండు entries** గా రాయబడుతుంది, మరియు వాటి మొత్తం **తప్పక సున్నా**.

₹500 A నుంచి B కి: {A: -500} మరియు {B: +500}. మొత్తం = 0.

ఒక entry రాయబడకపోతే ఆ లావాదేవీ మొత్తం **సున్నా కాదు** – మరియు అది **ఆ లావాదేవీని పేరు పెట్టి** చెప్తుంది.

ఇది RAID యొక్క parity, లేదా ఒక checksum లాంటిది: **మీరు కొంచెం ఎక్కువ రాసి, దానికి బదులుగా "ఏది విరిగింది" అనే సమాచారం కొంటారు.**

```
// $5 – ఒక transaction యొక్క మొత్తం *తప్పక* సున్నా అవ్వాలి.  
// ఇది ఒక accounting సంప్రదాయం కాదు – ఇది ఒక దోష-గుర్తింపు సంకేతం.  
post(txnId, entries) {  
  if (this.#txns.has(txnId)) {  
    this.stats.duplicate++; return { ok: true, duplicate: true };  
  }  
}
```

```

if (!Array.isArray(entries) || entries.length < 2)
  return { ok: false, reason: 'TOO_FEW_ENTRIES' };
let sum = 0;
for (const e of entries) { ...; sum += e.amt; }
if (sum !== 0) {
  this.stats.rejectedUnbalanced++;
  return { ok: false, reason: 'UNBALANCED', sum };
}
...
}

```

మరియు రెండో భాగం – **balance ఒక column కాదు:**

```

// §5 – balance ఒక *నిల్వ చేసిన సంఖ్య* కాదు, entries నుంచి వచ్చేది.
// cache ఉంది, కానీ అది ఎప్పుడైనా తిరిగి లెక్కించదగినది.
balance(acct) { return this.#balances.get(acct) || 0; }
recompute(acct) {
  let s = 0;
  for (const e of this.#entries) if (e.acct === acct) s += e.amt;
  return s;
}

```

"అయితే **CACHE** ఎందుకు? అది మళ్ళీ నిల్వ చేసినట్టే కదా?"

తేడా ఇది: **cache ఒక ఉత్పన్నం, ఒక సత్యం కాదు.**

అది తప్పైతే మీరు `recompute()` నడిపి సరిచేయొచ్చు. మరియు §12 యొక్క నియమం 2 సరిగ్గా అదే పరీక్షిస్తుంది – **cache ఎప్పుడూ లెక్కించినదానితో సరిపోవాలి.**

Single-entry lo balance **సత్యం** – అది తప్పైతే సరిచేయడానికి ఏమీ లేదు. అదే మొత్తం తేడా.

ఒక వైఫల్యం · రెండు నిర్మాణాలు

single-entry

balance[A] -= 500 ✓

balance[B] += 500 ✗ వఫలం

₹500 పోయింది · ఎక్కడో తెలియదు

double-entry

{txn 42, A, -500} ✓

{txn 42, B, +500} ✗ వఫలం

txn 42 మొత్తం = -500 ≠ 0

రెండింటిలోనూ **అదే వైఫల్యం**, అదే డబ్బు నష్టం.

తేడా – double-entry ఆ నష్టాన్ని **ఒక పేరుతో** చెప్తుంది.

వెతకాల్సినవి: 50,000 → **30**

అదనపు ఖర్చు: ఒక్కో లావాదేవీకి ఒక అదనపు row.

Part 3 – రెండో విరుపు: వెనక్కి పూడ్చడం కూడా విఫలమవుతుంది

6. Step – saga • ముందుకి, లేదా వెనక్కి

ఒక చెల్లింపు నాలుగు services ని తాకుతుంది:

1. Wallet నుంచి డబ్బు పట్టుకో
2. Gateway lo charge చెయ్యి
3. నిల్వ తగ్గించు
4. Order పూర్తి చెయ్యి

ఇవి వేర్వేరు systems కాబట్టి ఒకే database transaction అసాధ్యం. ప్రామాణిక జవాబు **saga**: ముందుకి వెళ్తూ, ఏదైనా అడుగు విఫలమైతే **ఇప్పటివరకు చేసినవాటిని వెనక్కి పూడ్చడం**.

అది సరైన నమూనా. కానీ ఒక ప్రశ్న మిగిలిపోతుంది, మరియు చాలామంది దాన్ని అడగరు.

7. రెండో విరుపు – 1,135 చెల్లింపులు మధ్యలో ఆగిపోయాయి

వెనక్కి పూడ్చడం కూడా ఒక **distributed operation**. అది కూడా విఫలం కావచ్చు.

2,00,000 చెల్లింపులు • ఒక్కో అడుగు విఫలమయ్యే అవకాశం 2%

అడుగులు | compensate విఫలం | పూర్తయినవి | శుభ్రంగా వెనక్కి | ▲ చిక్కుకున్నవి

2		0%		96.1%		7,891		0 (0.00%)
2		2%		96.1%		7,809		82 (0.04%)
4		2%		92.2%		15,032		508 (0.25%)
6		2%		88.6%		21,708		1,135 (0.57%)
6		10%		88.6%		17,805		4,983 (2.49%)

"చిక్కుకున్నవి" అంటే ఏమిటో స్పష్టంగా చెప్పాలి

అవి విఫలమైన చెల్లింపులు కాదు. అవి సగం జరిగిన చెల్లింపులు.

Wallet నుంచి డబ్బు పోయింది, కానీ order లేదు. లేదా order ఉంది, కానీ నిల్వ తగ్గలేదు.

ఆరు అడుగులు, 2% compensation వైఫల్యంతో – **1,135 చెల్లింపులు**. రోజుకి 50 లక్షల చెల్లింపుల స్థాయిలో అది **రోజుకి 28,000**, ప్రతి ఒక్కటీ ఒక మనిషి చూడాల్సినది.

మరియు గమనించండి: అడుగులు 2 నుంచి 6 కి పెరిగితే చిక్కుకునేవి **82 నుంచి 1,135** – 14 రెట్లు.

8. Step – పూడ్చడాన్ని వదిలేయకూడదు

పరిష్కారం compensation ని **మెరుగుపరచడం** కాదు – అది ఎప్పుడూ విఫలమవుతుంది. పరిష్కారం దాన్ని **వదిలేయకపోవడం**.

కానీ మళ్ళీ ప్రయత్నించాలంటే ఒక షరతు ఉంది: **compensation idempotent అయి ఉండాలి**. లేకపోతే రెండోసారి ప్రయత్నించడం డబ్బుని రెండుసార్లు వెనక్కి ఇస్తుంది.

అది LLD Deep 21 §5 lo నిరూపించాం. ఇక్కడ దాన్ని **వాడతాం**:

```
// §8 – compensation తప్పక idempotent • txnId ద్వారా
compensate(txnId) {
  const idx = this.#txns.get(txnId);
  if (!idx) { this.stats.compFailed++; return { ok: false, reason: 'NO_TXN' }; }
  const revId = `rev:${txnId}`;
  const rev = idx.map(i =>
    ({ acct: this.#entries[i].acct, amt: -this.#entries[i].amt }));
  const r = this.post(revId, rev);
  if (r.ok) this.stats.compensated++;
  return r;
}
```

`rev:${txnId}` అనేది ఒక **ఉత్పన్న key** – అదే లావాదేవీకి ఎప్పుడూ అదే reversal id. కాబట్టి ఎన్నిసార్లు ప్రయత్నించినా **ఒక్కసారే** వర్తిస్తుంది.

మరియు reversal కూడా ఒక **సాధారణ double-entry లావాదేవీ** – అది కూడా సున్నాకి సమానం, కాబట్టి §5 యొక్క పరీక్ష దానికి వర్తిస్తుంది.

పరిష్కారం: compensation ని idempotent చేసి మళ్ళీ ప్రయత్నించడం
(6 అడుగులు • compensate విఫలం 10%)

ఎన్నిసార్లు మళ్ళీ | చిక్కుకున్నవి | retry అవసరమైనవి

0	4,983 (2.491%)	4,983
1	566 (0.283%)	4,879

3		5 (0.003%)		4,929
6		0 (0.000%)		4,926

2.491% → 0% • కేవలం "వదిలేయకపోవడం" వల్ల

ఆరుసార్లు మళ్ళీ ప్రయత్నిస్తే ఒక్కటే చిక్కుకోలేదు.

మరియు ఖర్చు చూడండి – **retry అవసరమైనవి 4,926**, అంటే మొత్తం చెల్లింపుల్లో 2.5%. మిగతా 97.5% మొదటి ప్రయత్నంలోనే పూర్తయ్యాయి.

ఖరీదైనది retry కాదు – retry ని సాధ్యం చేసే idempotency. అది లేకపోతే మళ్ళీ ప్రయత్నించడం పరిష్కారం కాదు, ఒక కొత్త bug.

Part 4 – మూడో విరుపు: gateway చెప్పేది క్రమంలో రాదు

9. Step – webhooks వింటూ స్థితి మార్చడం

Payment gateway మీ system కి **webhooks** పంపుతుంది:

authorized → captured → settled

సహజమైన handler:

```
app.post('/webhook', (req) => {  
  payments.update(req.paymentId, { state: req.state });  
});
```

వచ్చినదాన్ని వర్తింపజేయడం. సూటిగా ఉంది.

కానీ ఆ webhooks **HTTP మీద** వస్తాయి, వేర్వేరు కనెక్షన్ మీద, retries తో. అంటే:

- **క్రమం తప్పొచ్చు** – captured కంటే ముందు settled రావొచ్చు
- **నకలు రావొచ్చు** – ప్రతి gateway "కనీసం ఒకసారి" హామీ ఇస్తుంది, "సరిగ్గా ఒకసారి" కాదు
- **చాలా ఆలస్యం కావొచ్చు** – ఒక retry గంట తర్వాత రావొచ్చు

10. మూడో విరుపు – 12.5% తప్పు స్థితిలో ఆగిపోతాయి

1,00,000 చెల్లింపులు • webhooks 15% క్రమం తప్పితాయి • 20% నకలు వస్తాయి

విధానం	తప్పు తుది స్థితి	నకలు/వెనక్కి వెళ్ళినవి
వచ్చినదాన్ని వర్తింపజేయడం	12,466 (12.5%)	49,459
క్రమ సంఖ్య చూసి ముందుకే	0 (0.0%)	82,502

12,466 చెల్లింపులు SETTLED అయినా మే SYSTEM LO CAPTURED గా ఉన్నాయి

అంటే: డబ్బు మే ఖాతాలో పడింది, కానీ మే system అది ఇంకా రాలేదని అనుకుంటోంది.

పర్యవసానాలు: వ్యాపారికి చెల్లింపు ఆగిపోతుంది, dashboard తప్పు చూపిస్తుంది, reconciliation విఫలమవుతుంది, మరియు ఎవరో ఒకరు **మళ్ళీ charge చేస్తారు**.

మరియు ఇది ఒక అరుదైన సంఘటన కాదు – **ఎనిమిదింటిలో ఒకటి**.

క్రమం ఎంత తప్పితే ఎంత నష్టం? (వచ్చినదాన్ని వర్తింపజేస్తే)

క్రమం తప్పే రేటు తప్పు తుది స్థితి	
2%	1.6%
5%	4.1%
15%	12.5%
30%	25.7%

నష్టం క్రమం తప్పే రేటుతో **సరళరేఖలో** పెరుగుతుంది. అంటే ఇది "అరుదైన అంచు పరిస్థితి" కాదు – **ఇది మే network ఎంత అస్తవ్యస్తంగా ఉందో దాని నేరుగా ప్రతిబింబం**.

11. Step – స్థితి ఎప్పుడూ వెనక్కి పోకూడదు

పరిష్కారం సులభం, మరియు అది HLD Deep 04 §5 lo చూసిన అదే ఆలోచన: **వచ్చిన క్రమాన్ని నమ్మకూడదు**.

```
// §11 – webhooks క్రమం తప్పి, నకలుగా వస్తాయి. స్థితి *ఎప్పుడూ వెనక్కి పోకూడదు*.
applyWebhook(paymentId, state, seq = null) {
  if (!(state in RANK)) return { ok: false, reason: 'UNKNOWN_STATE' };
  const cur = this.#payments.get(paymentId);
  if (!cur) {
    this.#payments.set(paymentId, { state, seq });
    this.stats.webhookApplied++; return { ok: true };
  }
  // seq ఉంటే దాన్ని, లేకపోతే స్థితి క్రమాన్ని నమ్ముతాం
  const goBack = seq !== null && cur.seq !== null
    ? seq <= cur.seq : RANK[state] <= RANK[cur.state];
  if (goBack) { this.stats.webhookStale++; return { ok: true, ignored: true }; }
  this.#payments.set(paymentId, { state, seq });
  this.stats.webhookApplied++;
  return { ok: true };
}
```

రెండు రక్షణలు, ఎందుకంటే GATEWAYS ఒకేలా ఉండవు

కొన్ని gateways ప్రతి webhook తో ఒక క్రమ సంఖ్య ఇస్తాయి – అది ఉత్తమం, దాన్ని వాడండి.

ఇవ్వని gateways కి, స్థితుల క్రమం తెలిసిన జ్ఞానం (authorized < captured < settled) ఆ పని చేస్తుంది.

రెండూ ఒకే నియమం అమలు చేస్తున్నాయి: ఒకసారి ముందుకి వెళ్ళక, వెనక్కి రాకూడదు. మరియు అది నకలులని కూడా ఉచితంగా పరిష్కరిస్తుంది – ఒకే స్థితి రెండోసారి వస్తే అది "ముందుకి" కాదు, కాబట్టి దాటేస్తాం.

Part 5 – పూర్తి system

12. మొత్తం code • 7.2 లక్షల entries • mutation testing

నాలుగు నియమాలు

#	నియమం	విరిగితే అర్థం
1	ప్రతి లావాదేవీ మొత్తం సున్నా	డబ్బు పుట్టింది లేదా మాయమైంది
2	balance cache ఎప్పుడూ recompute() తో సరిపోవాలి	ఉత్పన్నం సత్యంతో విడిపోయింది
3	మొత్తం ledger మొత్తం సున్నా	ఎక్కడో ఒక entry తప్పిపోయింది
4	చెల్లింపు స్థితి ఎప్పుడూ వెనక్కి పోకూడదు	§11 రక్షణ లేదు

నియమం 3 double-entry యొక్క అసలు శక్తి: ప్రతి entry ఎక్కడో ఒక వ్యతిరేక entry కలిగి ఉంటుంది కాబట్టి, అన్ని ఖాతాల మొత్తం ఎప్పుడూ సున్నా (బయటి డబ్బు కోసం ఒక EXTERNAL ఖాతా ఉంచితే). అది ఒక్క లెక్కతో మొత్తం system ఆరోగ్యాన్ని చెప్తుంది.

1,200 యాదృచ్ఛిక ప్రయోగాలు • 3,62,995 transactions • 7,25,990 entries
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

posted	3,62,995
duplicate	3,97,225
rejectedUnbalanced	48,611
entries	7,25,990
webhookApplied	1,42,198
webhookStale	1,28,115
compensated	85,601

రెండు సంఖ్యలు గమనించండి: duplicate 3,97,225 – అంటే idempotency 4 లక్షల సార్లు పని చేసింది, మరియు webhookStale 1,28,115 – §11 యొక్క రక్షణ 1.28 లక్షల సార్లు ఒక పాత webhook ని ఆపింది.

Test విఫలం కాగలదా? – నాలుగు మార్పులు

మార్పు లేని code	→ 0/1200 విఫలం
సమతుల్యత పరీక్ష తీసేస్తే	→ 400/400 విఫలం
ఉదా: సమతుల్యం కాని txn t161 అంగీకరించబడింది	
idempotency పరీక్ష తీసేస్తే	→ 400/400 విఫలం
ఉదా: నకలు txn t210 balance ని మార్చింది	
webhook వెనక్కి వెళ్ళనివ్వడం	→ 400/400 విఫలం
ఉదా: p24: స్థితి వెనక్కి పోయింది failed → refunded	

balance cache ని entries తో కలిపి నవీకరించకపోతే → 400/400 విఫలం
ఉదా: a: cache 0 ≠ లెక్కించినది 282

MUTATION TESTING ఒక చచ్చిన పంక్తి ని బయటపెట్టింది - మరియు నేను దాన్ని తీసేశాను

నా `compensate()` lo ఒక idempotency పరీక్ష ఉండేది: `if (this.#txns.has(revId)) return duplicate.`

దాన్ని తీసేసి పరీక్షిస్తే **0/400 విఫలం** - అంటే అది ఏమీ చేయడం లేదు.

కారణం: `post(revId, ...)` **అదే పరీక్షని** ఇప్పటికే చేస్తుంది. ఒకే రక్షణ రెండు చోట్లు.

"జాగ్రత్త కోసం" అని ఉంచిన ఒక అనవసరమైన పరీక్ష చదివేవాడిని **అది అవసరమని** నమ్మిస్తుంది. LLD Deep 26 lo ఇదే జరిగింది, మరియు అప్పుడూ దాన్ని తీసేశాను.

Part 6 – Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-5	"ఇక్కడ అసలు ప్రశ్న గుర్తింపు, నివారణ కాదు" – ఇది మిమ్మల్ని వేరుగా చూపిస్తుంది
5-9	§2 ప్రశ్నలు. ముఖ్యంగా "compensation విఫలమైతే?"
9-20	మొదటి విరుపు. Double-entry ని ఒక checksum గా వివరించండి. 50,000 → 30
20-30	రెండో విరుపు. Saga → 1,135 చిక్కుకున్నవి. Idempotent retry → 0
30-38	మూడో విరుపు. Webhooks → 12.5%. "స్థితి వెనక్కి పోకూడదు"
38-45	నాలుగు నియమాలు, ముఖ్యంగా "ledger మొత్తం సున్నా"

ఏమి తప్పక చెప్పాలి

1. Double-entry ఒక సంప్రదాయం కాదు, ఒక దోష-స్థాన-నిర్ధారణ సంకేతం. 50,000 → 30 records.
2. Balance ఒక column కాదు, ఒక ఉత్పన్నం. తప్పితే తిరిగి లెక్కించొచ్చు.
3. Compensation కూడా విఫలమవుతుంది – saga ఒక్కటే సరిపోదు, దానికి retry కావాలి.
4. Retry కి idempotency ఒక పూర్వ షరత్తు, ఒక అదనపు feature కాదు.
5. వచ్చిన క్రమాన్ని ఎప్పుడూ నమ్మకూడదు. స్థితి ఎప్పుడూ ముందుకే.

ఏమి వదిలేయాలి

- Idempotency key యొక్క అమలు వివరాలు – LLD Deep 21 lo ఉన్నాయి
- PCI compliance, card tokenisation – ఒక వాక్యం
- Fraud detection – ఇది ఒక ML problem
- Currency conversion rates – §15 lo ఒక వాక్యం

14. నోటీస్ చెప్పాల్సిన English script

"In most systems a lost record is an annoyance. Here it's money, so I want to start somewhere unusual: not with **preventing** errors, but with **detecting** them. Because the dangerous failure in payments isn't a crash – it's a silent one, where ₹11,000 is missing and nobody knows which transaction lost it."

"The naive schema is a balance column per account and two updates per transfer. In one database that's safe. But in a real payment system the payer's wallet, the merchant ledger and the commission account are **different services**, so there's no single transaction, and anything can happen between the two writes."

"I simulated 50,000 operations with deposits and payouts flowing in and out, and failed 0.1% of the second writes. Both designs told me something was wrong. Only one told me **where**. With single-entry I know the total doesn't reconcile – and to find it I'd audit all 50,000 operations across several systems. With double-entry, each transaction's entries must sum to zero, so a missing entry names **exactly 30 transaction ids**. That's **1,667x less** to search."

"I'd add that my measurement was generous to single-entry: it detected the discrepancy only because I tracked deposits and payouts perfectly. In production those come from bank statements and gateway reports with their own lag and errors, so realistically single-entry can't reliably detect even the total. **It lost even with the scales tipped in its favour.**"

"So: **double-entry isn't accounting tradition, it's an error-locating code** – the same idea as parity or a checksum. You write one extra row and buy the ability to name what broke. And balance stops being a stored truth and becomes a derived value; I keep a cache, but it's recomputable, and one of my invariants is that the cache always equals the recomputation."

"Second, the saga. A payment touches four to six services, so you go forward and compensate on failure. The question people skip is: **what if the compensation fails?** It's a distributed call too. I measured six steps with 2% step failure and 2% compensation failure: **1,135 of 200,000 payments end up stuck** – money taken but no order, or the reverse. At five million payments a day that's about 28,000 daily cases, each needing a human. And it scales badly: going from two steps to six took stuck payments from 82 to 1,135."

"The fix isn't better compensation – it'll always fail sometimes. It's **not giving up**: retry until it succeeds. Six retries took 2.49% stuck to **zero**. But retrying is only safe if compensation is **idempotent**, which is why I derive the reversal id from the original – `rev: txn42` – so it applies exactly once no matter how many attempts. **Idempotency isn't an extra feature here, it's the precondition that makes retry legal.**"

"Third, webhooks. Gateways send authorized, captured, settled over HTTP with retries, so they arrive out of order and duplicated – every gateway promises at-least-once, not exactly-once. Applying whatever arrives left **12.5% of payments in the wrong final state**: settled at the gateway, captured in my system, which means the merchant isn't paid, reconciliation fails, and eventually someone charges the card again. And it's linear in reorder rate – 2% reordering gives 1.6% wrong, 30% gives 25.7%, so it's not an edge case, it's a direct reflection of how messy your network is."

"The fix is the same idea as ordering chat messages: **never trust arrival order**. State moves forward only – by sequence number if the gateway provides one, otherwise by known state rank. That handles duplicates for free, since a repeat isn't forward."

"Four invariants, 1,200 runs, 726,000 entries, zero violations, all four mutations caught. The nicest invariant is that the **whole ledger sums to zero** – with an EXTERNAL account for money entering and leaving, one number tells you the entire system is intact. And mutation testing earned its keep again: it showed an idempotency check in my compensate path was dead code, because `post()` already performs the same check. I deleted it – a redundant guard teaches the next reader that it's necessary."

15. Follow-ups – currencies, refunds, reconciliation

"డబ్బుని ఏ రకంలో నిల్వ చేయాలి?"

Float ఎప్పుడూ కాదు. `0.1 + 0.2 !== 0.3` – మరియు §12 యొక్క నియమం 1 ("మొత్తం సరిగ్గా సున్నా") float తో ఎప్పటికీ నెగ్గదు. ఆచరణ: **అతి చిన్న యూనిట్‌లో పూర్ణాంకాలు** (ఫైసలు, cents). JavaScript lo పెద్ద మొత్తాలకి `BigInt`. మరియు ప్రతి entry తో **currency code** ఉండాలి – మరియు **వేర్వేరు currencies ఉన్న entries ఒకే transaction lo కలవకూడదు**; మార్పిడి ఒక ప్రత్యేక లావాదేవీ, ఒక `FX` ఖాతా ద్వారా.

"Refund ఎలా – compensation లాంటిదేనా?"

కాదు, మరియు ఈ తేడా ముఖ్యం. Compensation అంటే "ఇది జరగకూడదు, దాన్ని చెరిపేయ్". Refund అంటే "ఇది జరిగింది, ఇప్పుడు ఒక కొత్త వ్యతిరేక లావాదేవీ". Ledger lo compensation మరియు refund రెండూ కొత్త entries గానే వస్తాయి (ఎప్పుడూ పాతవాటిని తీసేయం), కానీ వాటి **అర్థం** వేరు – మరియు పన్ను, reporting, వివాదాల కోసం ఆ తేడా అవసరం. **Ledger append-only, ఎప్పుడూ update కాదు.**

"Reconciliation ఎలా చెయ్యాలి?"

Gateway రోజూ ఒక settlement file ఇస్తుంది. మీ ledger తో దాన్ని సరిపోల్చాలి. §5 దీన్ని చాలా సులభం చేస్తుంది: ప్రతి gateway లావాదేవీకి మీ దగ్గర ఒక `txnId` ఉంది, మరియు రెండు వైపులా మొత్తాలు సరిపోవాలి. తేడాలు మూడు రకాలు – **మీ దగ్గర ఉంది, వాళ్ళ దగ్గర లేదు** (బహుశా webhook రాలేదు), **వాళ్ళ**

దగ్గర ఉంది, మీ దగ్గర లేదు (మీరు కోల్పోయారు), రెండింటిలోనూ ఉంది కానీ మొత్తం వేరు (అత్యంత ఆందోళనకరం). LLD Deep 21 §13 lo `reconcile()` ఇదే.

"ఒక ఖాతా **balance** ని వేగంగా ఎలా చదవాలి – **entries** కోట్లు ఉంటే?"

§5 lo `cache` ఉంది, కానీ అది `memory` lo. నిజమైన `scale` lo: ఒక **snapshot** పద్ధతి – ప్రతి రోజు చివర ఒక `balance_snapshot(acct, date, amount)` రాయడం, మరియు `balance = చివరి snapshot + ఆ తర్వాతి entries`. అప్పుడు `recompute()` కోట్ల `rows` కాకుండా ఒక రోజు `rows` మాత్రమే చూస్తుంది – **మరియు అది ఇంకా ఒక ఉత్పన్నమే**, ఒక సత్యం కాదు.

"రెండు చెల్లింపులు ఒక ఖాతాని ఒకసారి తాకితే?"

Ledger append-only కాబట్టి **రెండూ రాయొచ్చు** – వాటి మధ్య పోటీ లేదు. పోటీ ఉన్నది ఒక్క చోట: **"balance సరిపోతుందా?"** అనే పరీక్ష. దానికి ఒక `conditional insert` కావాలి (`WHERE balance >= amt`), లేదా ఆ ఖాతా మీద ఒక `serialised` దారి. ఇదే **single-entry vs double-entry** యొక్క ఆఖరి తేడా: **single-entry lo ప్రతి write ఒక పోటీ; double-entry lo పోటీ ఒక్క తనిఖీ దగ్గరే**.

16. ఏమి నేర్చుకున్నాం

1. చెల్లింపు **systems** lo అసలు ప్రశ్న నివారణ కాదు, గుర్తింపు. Crash ని ఆపలేరు. కానీ ఏమి విరిగిందో పేరు పెట్టి చెప్పగలగడం మీ చేతిలో ఉంది.

2. **Double-entry** ఒక **accounting** సంప్రదాయం కాదు – అది ఒక దోష-స్థాన-నిర్ధారణ సంకేతం. ఒక అదనపు `row` కి బదులుగా "ఏ లావాదేవీ విరిగింది" అనే సమాధానం. **50,000 → 30, 1,667** రెట్లు.

3. **Balance** ఒక సత్యం కాకూడదు, ఒక ఉత్పన్నం కావాలి. ఉత్పన్నం తప్పితే తిరిగి లెక్కించొచ్చు; సత్యం తప్పితే సరిచేయడానికి ఏమీ లేదు.

4. **Compensation** కూడా ఒక **distributed operation** – అది కూడా విఫలమవుతుంది. ఆరు అడుగులతో **1,135 చెల్లింపులు** ఎటూ కాని స్థితిలో. Saga ఒక్కటే సరిపోదు.

5. "వదిలేయకపోవడం" ఒక పరిష్కారం – కానీ దానికి **idempotency** ఒక పూర్వ ప్రరక్ష. **2.491% → 0%**, ఆరు `retries` తో. `Idempotency` లేకుండా `retry` ఒక పరిష్కారం కాదు, ఒక కొత్త bug.

6. వచ్చిన క్రమాన్ని ఎప్పుడూ నమ్మకూడదు. Webhooks 15% క్రమం తప్పితే **12.5% చెల్లింపులు** తప్పు స్థితిలో. మరియు నష్టం క్రమం తప్పే రేటుతో **సరళరఖలో** పెరుగుతుంది.

7. ఒక్క లెక్క మొత్తం **system** ఆరోగ్యాన్ని చెప్పగలదు. `EXTERNAL` ఖాతాతో కలిపి, అన్ని ఖాతాల మొత్తం ఎప్పుడూ సున్నా. అది విరిగితే ఏదో పోయింది – మరియు ఏ లావాదేవీలో అనేది వెంటనే తెలుస్తుంది.

8. అనవసరమైన రక్షణ ఒక అబద్ధం చెప్తుంది. నా `compensate()` lo ఉన్న `idempotency` పరీక్ష **0/400** – ఎందుకంటే `post()` అదే చేస్తోంది. దాన్ని ఉంచితే, చదివేవాడు అది అవసరమని నమ్ముతాడు.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ system యొక్క లక్ష్యం "ఎప్పుడూ తప్పు జరగకూడదు" కాదు – అది అసాధ్యం. లక్ష్యం "ప్రతి తప్పు తనని తాను ప్రకటించుకోవాలి".

§5 lo ప్రతి విరిగిన లావాదేవీ తన id తో అరుస్తుంది.

§8 lo ప్రతి అసంపూర్ణ saga ఒక retry queue lo కూర్చుంటుంది.

§11 lo ప్రతి పాత webhook లెక్కపెట్టబడుతుంది.

మౌనంగా విఫలమయ్యే చెల్లింపు system, విఫలం కాని దానికంటే ప్రమాదకరమైనది – ఎందుకంటే అది మిమ్మల్ని నమ్మకంలో ఉంచుతుంది.